Grupo
Bruno Gama Masseiras - RM: 562526
Kauai Rosa de Assis Rocha - RM: 563256
Leandro Afonso Silva Santos Junior - RM: 561344
Lucas Gilmar Pedro da Silva - RM: 566231
Murilo da Silva Basto - RM: 565445
Global Solution 2026 - CitizenTech | Domain Driven Design - Java

SafeRoute - API REST e trajetos alternativos

1. Proposta, justificativa e objetivos

Durante chuvas intensas, um caminho curto pode ficar impraticável por alagamento ou bloqueio. A SafeRoute centraliza relatos e condições dos trechos para demonstrar como uma aplicação pode apresentar alternativas de circulação, incluindo restrições de acessibilidade.

O território de referência é o Jardim Pantanal, no distrito Jardim Helena, em São Paulo (SP). A Prefeitura registra inundações recorrentes nessa região da várzea do Tietê. A rede de seis pontos e oito trechos é simulada: as coordenadas organizam o desenho e não representam ruas ou distâncias medidas. A validação com moradores permanece como etapa futura.

1.1 Objetivo e público

Implementar uma capacidade central da CitizenTech em Java: cadastrar a rede e seus sensores, registrar ocorrências, apresentar alertas e calcular uma alternativa que respeite os bloqueios conhecidos. O público proposto reúne moradores, trabalhadores e pessoas com mobilidade reduzida.

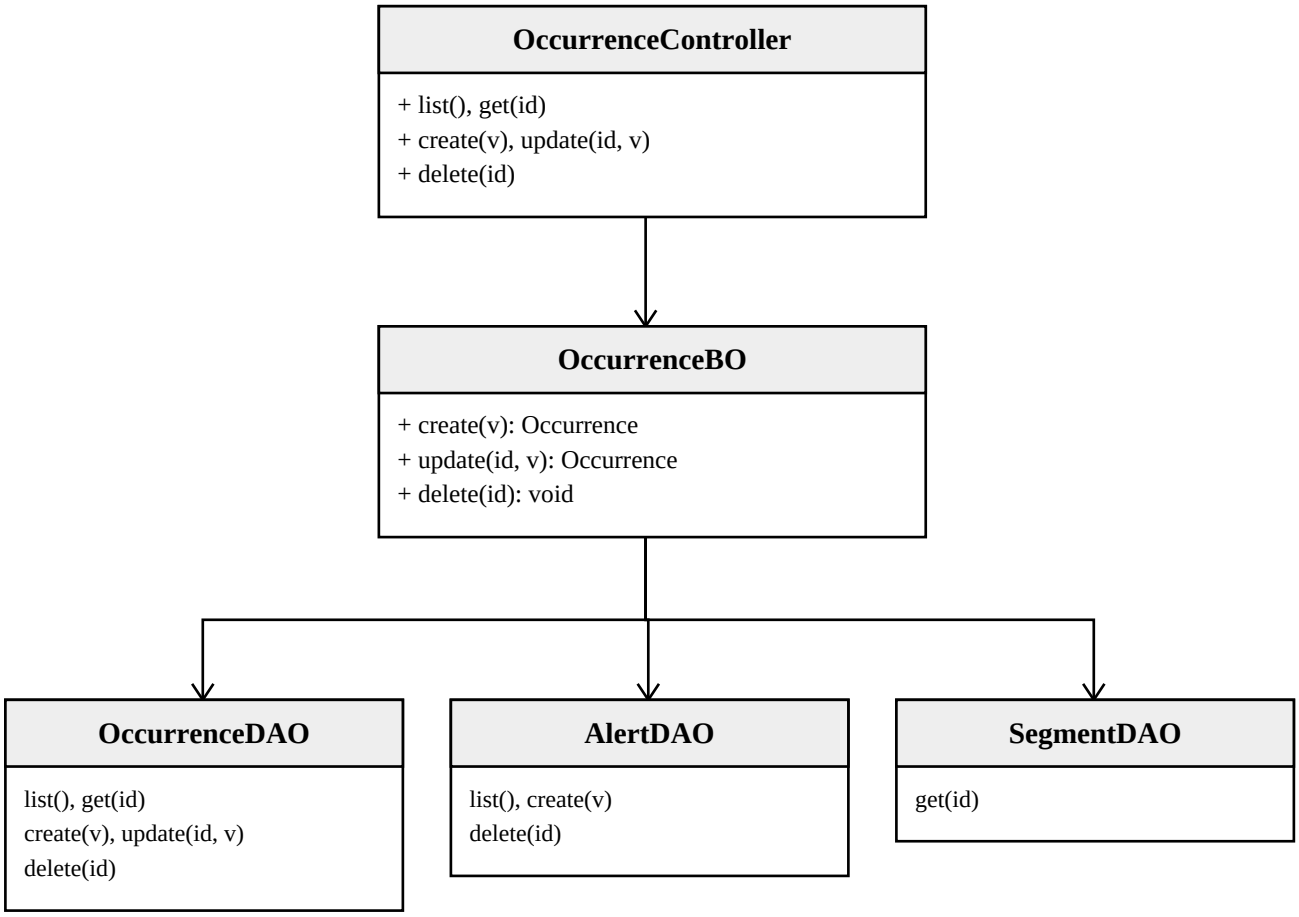
Funcionalidade	Comportamento implementado
Cadastro da rede	CRUD de regiões, pontos, trechos e sensores.
Monitoramento	Leituras simuladas e ocorrências com tipo, severidade e validade.
Consulta de rotas	Dijkstra em rede bidirecional, com restrições e custo ponderado.
Alertas	Consulta de ocorrências vigentes na interface web.
Integração	API REST + Oracle FIAP; grafo JSON disponível à equipe Python.

1.2 Justificativa e contribuição proposta

A combinação de ocorrências e acessibilidade permite avaliar alternativas antes de priorizar distância. A proposta se relaciona aos ODS 11, 10 e 13. O impacto poderá ser avaliado pela proporção de trajetos que evitam bloqueios e pelo tempo para localizar uma alternativa; não houve medição de impacto real.

2. Arquitetura e classes de aplicação

Controllers recebem HTTP; BOs validam relações e aplicam regras; DAOs executam SQL parametrizado. O modelo usa records e Bean Validation. O exemplo abaixo representa as dependências reais do fluxo de ocorrências.

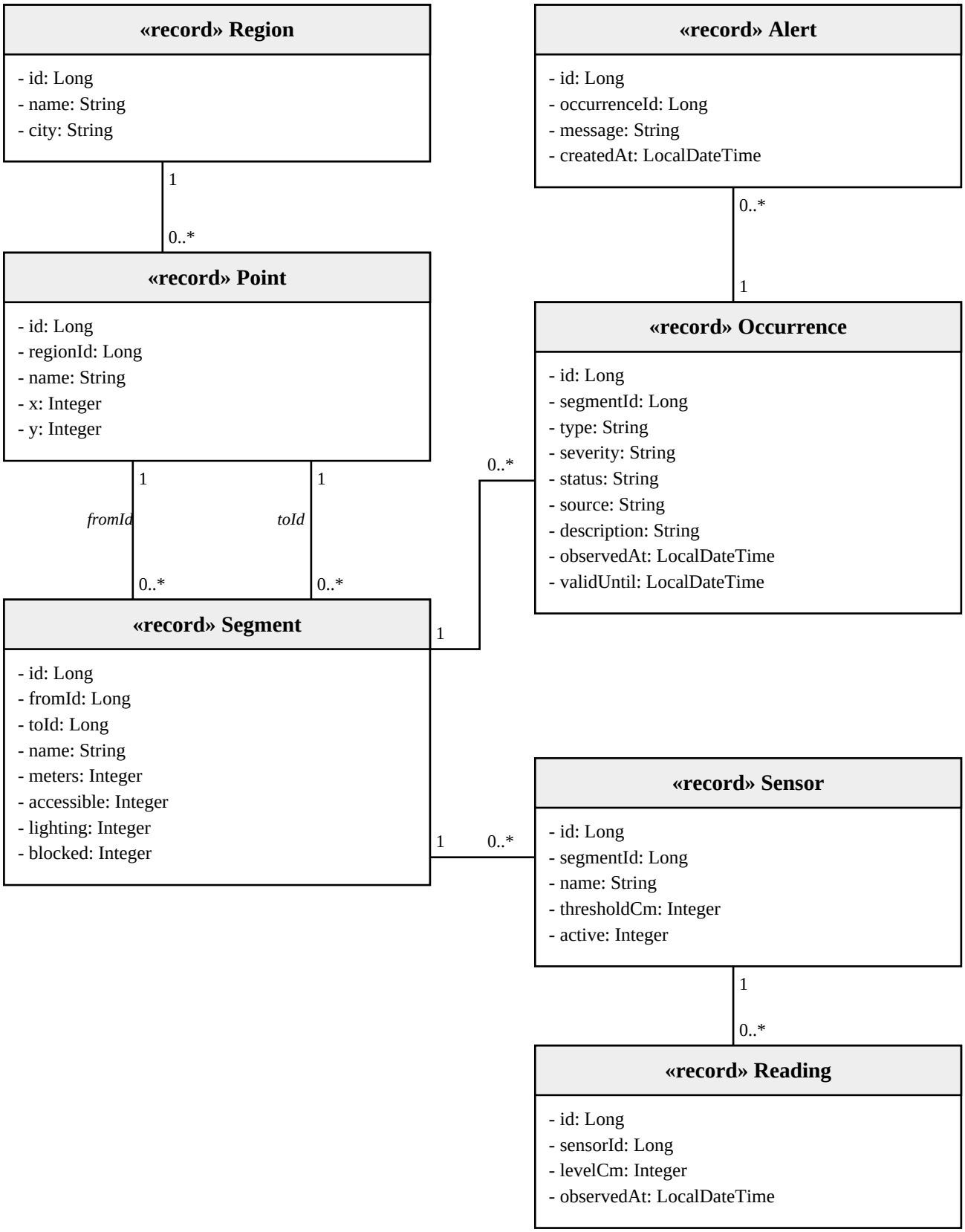


Setas de dependência: a controller delega ao BO; o BO acessa os três DAOs por injeção de construtor. Em criação e exclusão, @Transactional mantém ocorrência e alertas na mesma operação.

Fluxo	Classes correspondentes
Cadastros	Region / Point / Segment / Sensor + Controller, BO e DAO de cada tipo.
Leituras e alertas	MonitoringController → ReadingBO, ReadingDAO, AlertDAO e DAOs do grafo.
Trajetos	RouteController → RouteBO → PointDAO, SegmentDAO e OccurrenceDAO.
Infraestrutura	ApiAccess: chave de operador. ApiErrors: tratamento centralizado. Missing e Conflict: exceções de domínio.

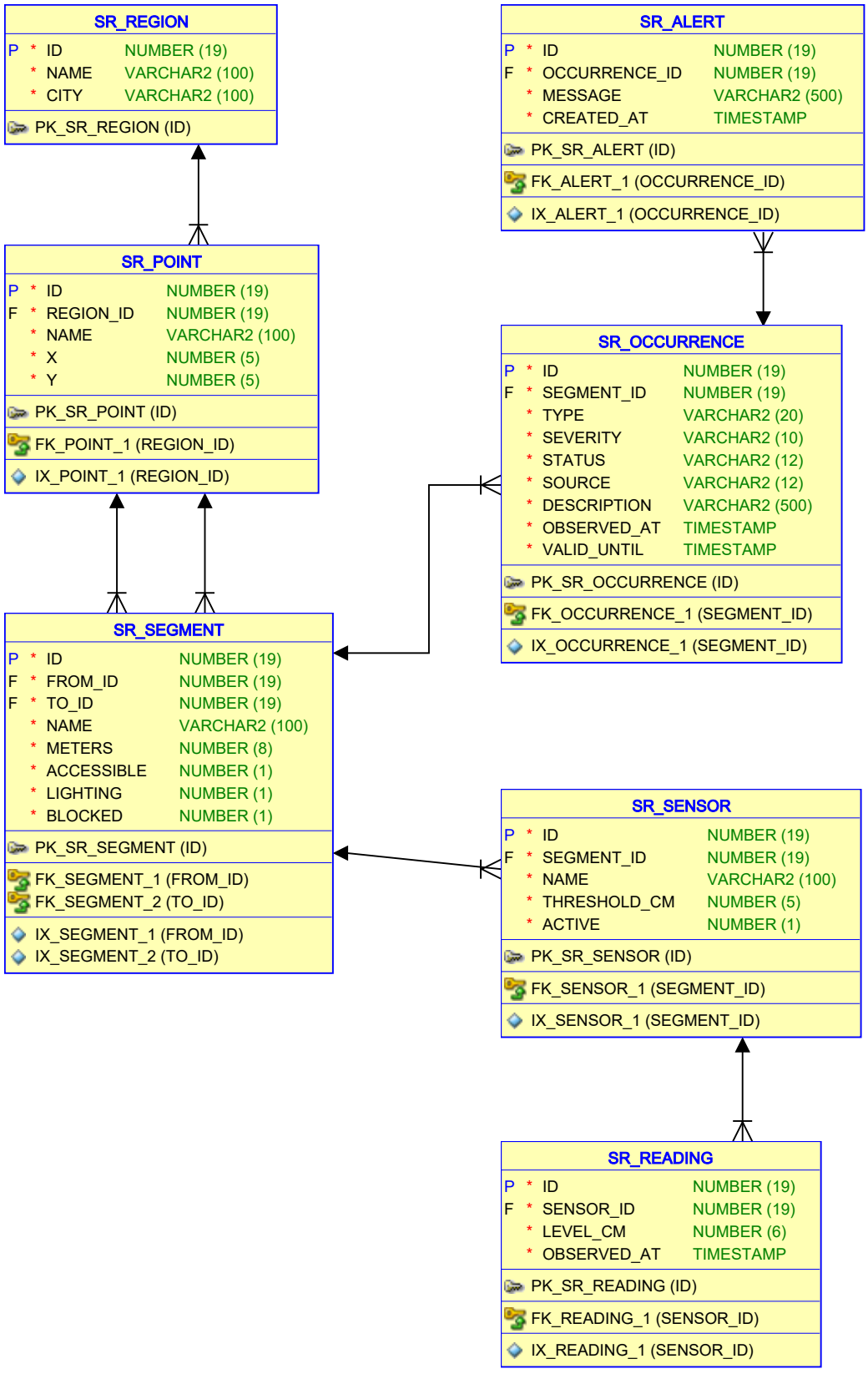
3. Diagrama de classes: domínio

Classes do pacote br.com.saferoute, implementadas como records imutáveis. As associações abaixo são representadas por identificadores (Long) no código e por FKs no Oracle.



4. Modelo relacional

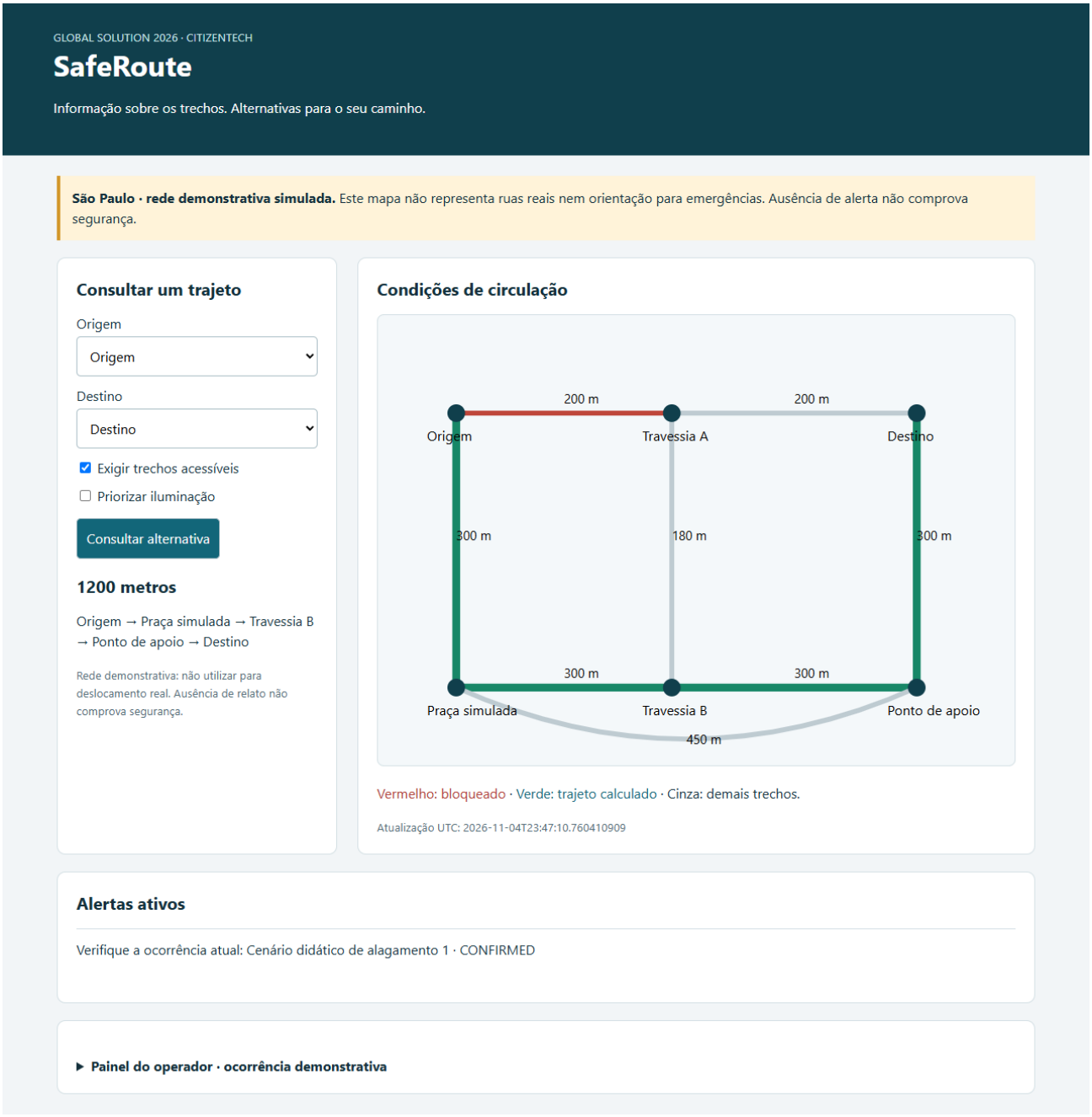
Modelo relacional completo, importado e exportado no Oracle SQL Developer Data Modeler 24.3.1. P = chave primária; F = chave estrangeira; * = atributo obrigatório.



Cada filho tem um pai obrigatório; cada pai pode ter 0..N filhos. Origem e destino são FKs distintas.

5. Interface: consulta de trajetos

Captura da interface publicada no Render, consumindo dados do Oracle FIAP. A origem e o destino são selecionados pelo usuário; a alternativa aparece no mapa e em texto.

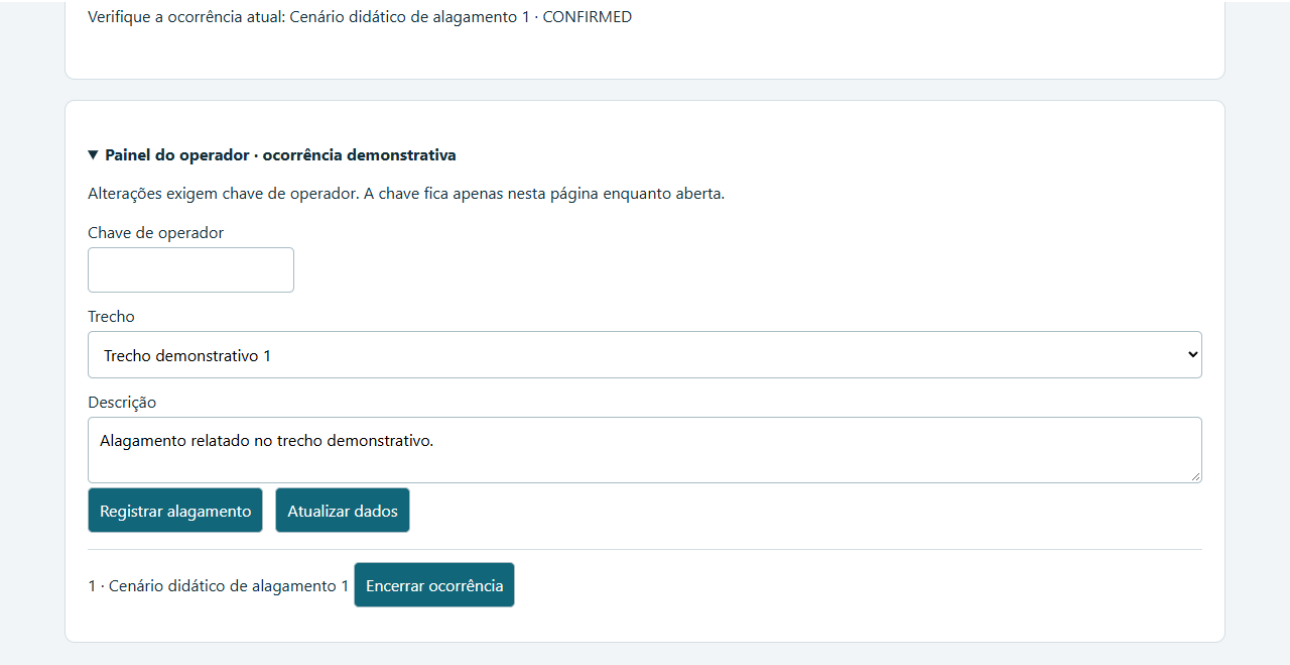


Cenário verificado: o trecho 1 está alagado e o perfil exige acessibilidade. O resultado percorre os pontos 1 → 4 → 5 → 6 → 3, com 1.200 metros. O trecho bloqueado permanece em vermelho; os trechos usados aparecem em verde.

O desenho é esquemático e mantém os avisos de simulação visíveis. A ausência de relato não é apresentada como garantia de segurança.

6. Interface: operação e ocorrências

O painel permite registrar um alagamento demonstrativo e encerrar ocorrências. As alterações exigem a chave do operador; a consulta pública de trajetos não exige autenticação.



Ação na tela	Requisição e efeito
Carregar rede	GET /api/graph; lista pontos, trechos e ocorrências.
Consultar alternativa	POST /api/routes; retorna FOUND ou NO_ROUTE e avisos.
Registrar alagamento	POST /api/occurrences; persiste a ocorrência e o alerta.
Encerrar ocorrência	PUT /api/occurrences/{id} com status CLOSED; deixa de aparecer nos alertas ativos.
Atualizar dados	GET /api/graph e /api/alerts; exige nova consulta para recalcular o trajeto.

7. Endpoints e requisições HTTP

Base pública: https://saferoute-gs-2026.onrender.com. O corpo das requisições é JSON. Cadastros usam GET, POST, PUT e DELETE conforme a tabela.

Recurso	Métodos e caminhos	Retorno
Regiões	GET/POST /api/regions GET/PUT/DELETE /api/regions/{id}	Region
Pontos	GET/POST /api/points GET/PUT/DELETE /api/points/{id}	Point
Trechos	GET/POST /api/segments GET/PUT/DELETE /api/segments/{id}	Segment
Sensores	GET/POST /api/sensors GET/PUT/DELETE /api/sensors/{id}	Sensor
Ocorrências	GET/POST /api/occurrences GET/PUT/DELETE /api/occurrences/{id}	Occurrence
Monitoramento	GET/POST /api/readings GET /api/alerts	Reading / alertas
Integração	GET /api/graph GET /api/health	Grafo / UP
Trajetos	POST /api/routes	RouteBO.Result

Exemplo de consulta:

POST /api/routes
{"fromId":1,"toId":3,"accessible":true,"night":false}

Exemplo de cadastro de região:

POST /api/regions | X-Admin-Key: sua-chave
{"name":"Setor de teste","city":"São Paulo"}

GET e PUT retornam 200; POST de cadastro retorna 201 com Location; DELETE retorna 204. Erros: 400 entrada inválida; 401 chave ausente/incorreta; 404 registro inexistente; 409 integridade ou conflito; 503 banco indisponível; 500 falha inesperada.

8. Como executar e acessar

O código completo está em **src/**, dentro do ZIP, junto de pom.xml, Maven Wrapper, SQL e testes. Extraia a pasta e siga os passos abaixo.
Referência exigida pelo enunciado: https://github.com/TLean07/saferoute-gs-2026
Deploy: https://saferoute-gs-2026.onrender.com

Ferramenta	Versão / finalidade
JDK / Spring Boot	Java 21 / Spring Boot 3.3.5.
Maven Wrapper	3.9.11; incluído no código-fonte.
Oracle / SQLcl	Oracle FIAP 19c / SQLcl 26.2.
Data Modeler	24.3.1; projeto nativo incluído no ZIP de Database.
Docker / Render	Build Maven + imagem Temurin 21; serviço gratuito.
Editor e verificação	IntelliJ IDEA 2025.3 ou editor compatível; JUnit, H2 e Playwright.

8.1 Demonstração local

1. Extraia o ZIP e abra a pasta do projeto com JDK 21 configurado.
2. No PowerShell, execute os comandos abaixo.
3. Abra http://localhost:8080. O perfil demo usa H2 descartável.

```
.\mvnw.cmd test
.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=demo"
```

Chave local do painel: **demo-saferoute-local**. O arquivo INICIAR_DEMO.cmd também inicia a demonstração. Linux/macOS: use sh mvnw.

8.2 Oracle e Render

Execute sql/01_ddl.sql e sql/02_dml.sql uma vez, em namespace SR_ vazio. Defina DB_URL, DB_USER, DB_PASSWORD e ADMIN_API_KEY no ambiente; use jdbc:oracle:thin:@//oracle.fiap.com.br:1521/ORCL. A chave deve ter pelo menos 16 caracteres. Inicie sem o perfil demo para usar Oracle.

No Render, use Dockerfile, plano gratuito e health check /api/health. O deploy desta entrega foi verificado com o Oracle. Credenciais permanecem privadas; o avaliador pode testar alterações no perfil demo. O primeiro acesso após inatividade pode levar mais tempo.

9. Verificação e limites

Nível de verificação	Resultado observado em 04/11/2026
Testes locais	9 testes de integração HTTP passaram; zero falhas, erros ou testes ignorados. Banco H2 em modo Oracle.
API publicada + Oracle	Health UP, grafo com 6 pontos/8 trechos, rota acessível de 1.200 m, proteção 401, CRUD completo, 404 e proteção de FK com 409.
Banco via MCP SQLcl	7 tabelas, 39 registros; Q1, Q2 e Q3 executadas com resultados preservados em JSON.
Interface	Consulta de alternativa e painel de operador inspecionados no navegador. Capturas nas páginas 7 e 8.
Modelo Oracle	DDL importado sem erros; projeto salvo, reaberto e exportado no Data Modeler.

9.1 Restrições assumidas

A base é simulada e não contém dados de moradores. Não houve pesquisa de campo no Jardim Pantanal, estimativa de impacto real ou integração com sensores oficiais. O sistema não representa uma solução de emergência pronta para uso. Segurança do lugar é tratada somente pelas ocorrências e pela iluminação cadastradas, sem base de criminalidade.

Os pesos do cálculo são heurísticos. Trechos com bloqueios conhecidos são removidos antes da otimização. As ocorrências da carga inicial têm validade de 30 dias; o cenário precisa ser revisado antes de demonstrar após esse período. A operação de produção usa uma chave compartilhada, adequada ao protótipo; gestão individual de operadores é evolução futura.

9.2 Referências e rastreabilidade

Território: Prefeitura de São Paulo, Urbanismo Social Jardim Pantanal. Fonte oficial.

FIAP. GS CitizenTech 2026, orientações de Database Design e Domain Driven Design - Java, p. 31-33. Proposta SafeRoute fornecida pelo grupo. Render: documentação oficial de Docker e health checks.

Regras, evidências e instruções completas estão no README e em STATUS_ENTREGA.md. O contrato /api/graph mantém a integração com as outras disciplinas explícita, sem atribuir a esta entrega funcionalidades ainda não implementadas.